Computer Sciences faculties USTHB.
2026 - 2027

Embedded Software Development

Chapter 1 : Introduction to Embedded Systems

COURSE 1 : Introduction to Embedded Systems

Teacher : Dr. ABADA Lyes.
E-mail : abada.lyes@gmail.com

Course structure

1 **Introduction to Embedded Systems**
- → MCU, ARM, peripherals, sensors, communications

2 **Embedded Linux and C**
- → OS, processes, threads, memory, real-time

3 **Embedded Software Development Tools**
- → Compiler, debugger, simulator, JTAG, Flash

4 **Multitasking and Synchronization**
- → RTOS, tasks, scheduling, synchronization

5 **Smartphone Programming**
- → Android, GUI, storage, concurrency, networking

6 **Embedded Systems Design Methods**
- → co-design, SysML, MARTE…..

What will we learn?

In this course

We will learn:

- What an embedded system is.
- Why embedded systems are different from general-purpose computers.
- The history and evolution of embedded systems.
- Where embedded systems are used.
- Their main characteristics and constraints.
- The basic architecture of an embedded system.
- The role of microcontrollers.
- Sensors and actuators.
- Data acquisition and control.
- Communication interfaces.

WHAT IS AN EMBEDDED SYSTEM?

- **Embedded System**
- An **embedded system** is a computer-based system designed to perform one or a limited set of specific functions as part of a larger system.

It usually combines:

- Hardware
- Embedded software
- Sensors and/or actuators
- Communication interfaces
- A power supply

Simple definition

- **An embedded system is a computer integrated into a larger system to perform a specific function.**

WHAT IS AN EMBEDDED SYSTEM?

- Embedded systems are everywhere.

Examples

- Washing machine controller
- Microwave oven
- Digital camera
- Car engine controller
- Printer
- Smartwatch
- Router
- GPS receiver
- Drone flight controller
- Industrial robot
- Medical equipment
- Smart home devices

This course is not only about programming microcontrollers. We will study the complete embedded software development process.

Embedded System vs PC

A PC is designed to run many different applications.
An embedded system is usually designed for a specific purpose.

General-purpose computer	**Embedded system**
Many applications	Specific functions
General-purpose OS	Bare-metal / RTOS / embedded Linux
Large resources	Often limited resources
User-oriented	Function-oriented
Usually easy to upgrade	Often constrained
Keyboard/display common	May have no conventional UI





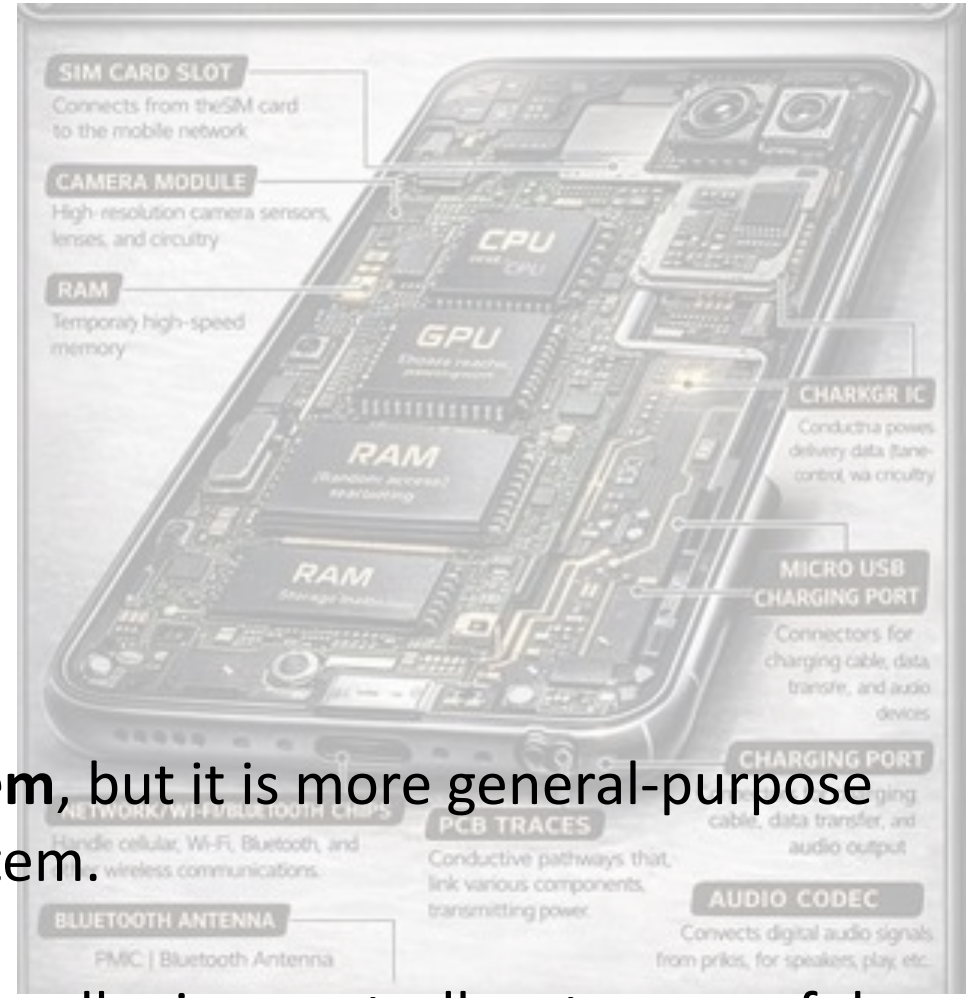


Important
The distinction is not absolute.
A powerful embedded system can run Linux and complex applications

Is a Smartphone an Embedded System?

A smartphone contains:

- CPU
- GPU
- RAM
- Storage
- Operating system
- Sensors
- Cameras
- Communication interfaces



It is a highly complex computing platform.

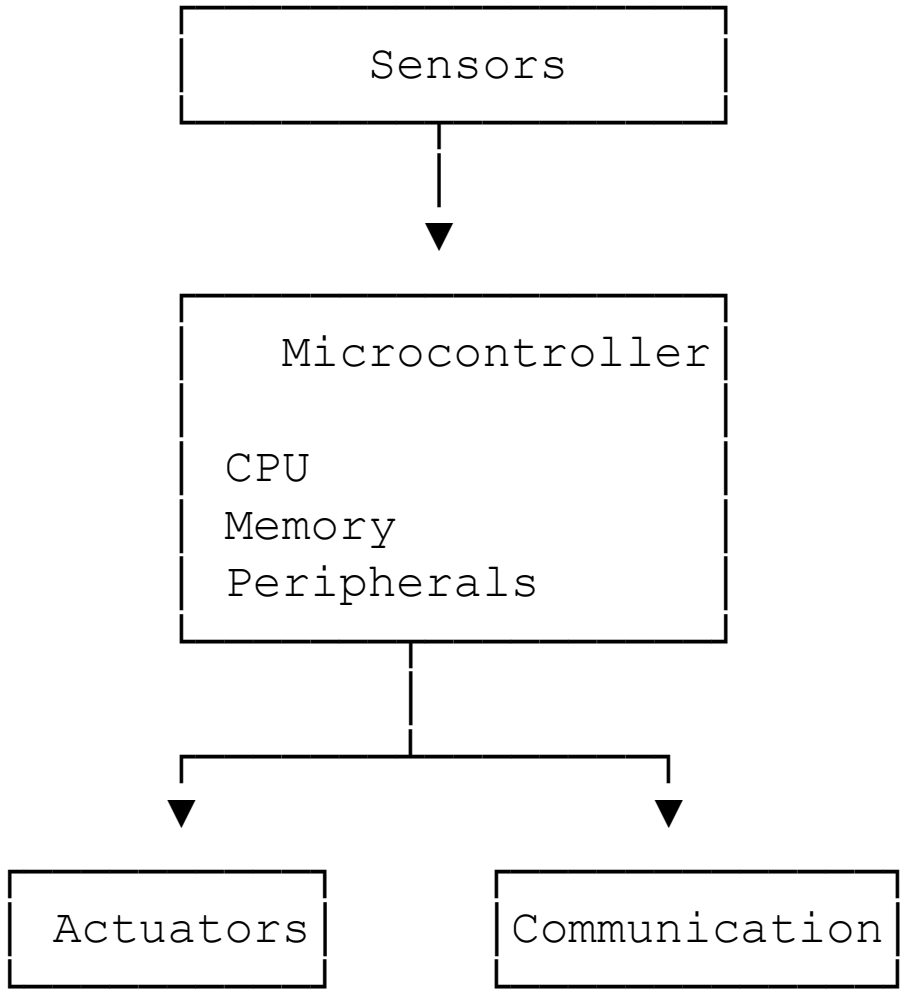
It can be considered an **embedded computing system**, but it is more general-purpose than a typical microcontroller-based embedded system.

Key idea

Embedded systems exist on a spectrum, from very small microcontrollers to powerful embedded computers.

GENERAL ARCHITECTURE

A simplified embedded system:



The microcontroller is often the central component.

Sensors collect information from the physical world.
This information goes to the microcontroller.
Inside the microcontroller, we have a CPU, memory and peripherals.

The CPU executes our software.
The memory stores the program and data.
The peripherals allow the CPU to communicate with external components.

So a very simple view is:
Sense → Process → Act → Communicate.

This simple architecture will appear again and again throughout this course.

Main Components

An embedded system may contain:

Processing unit

- Microcontroller (MCU)
- Microprocessor (MPU)
- System-on-Chip (SoC)

Memory

- Flash
- SRAM
- EEPROM
- External memory

Inputs

- Sensors
- Buttons
- Communication interfaces

Outputs

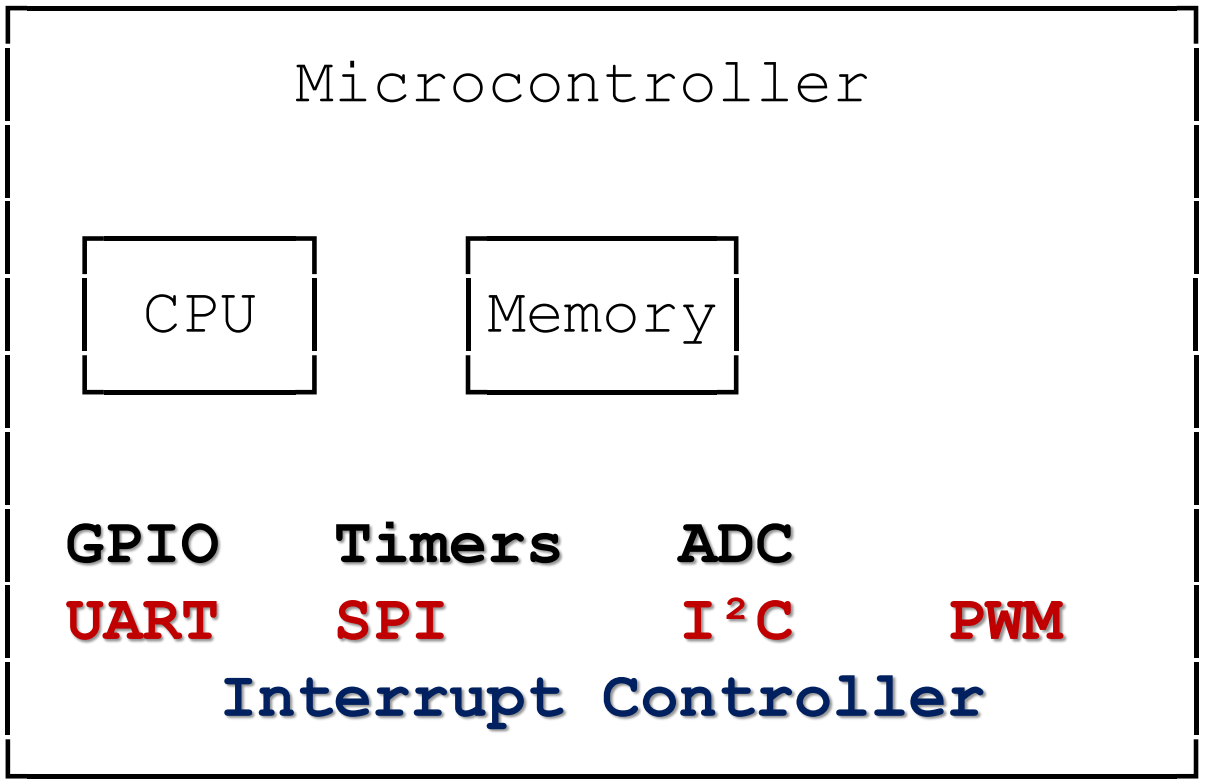
- Motors
- LEDs
- Displays
- Relays

Communication

- UART
- SPI
- I^2C
- CAN
- USB
- Ethernet
- Wi-Fi
- Bluetooth
- LoRa

What Is a Microcontroller?

A **microcontroller (MCU)** is an integrated circuit containing most of the components required to control an embedded system. Typically:



GPIO allows us to control digital input and output pins.
Timers allow us to measure time or generate periodic events.
An **ADC** allows us to convert analog signals into digital values.
UART, **SPI** and **I2C** allow communication with other devices.
PWM can be used, for example, to control motor speed or LED brightness.
And the **interrupt controller** allows the processor to react to events.

So one small chip can contain a complete computing system.

MCU vs MPU

Microcontroller

Usually integrates:

- CPU
- RAM
- Flash
- GPIO
- Timers
- Communication peripherals

Example:

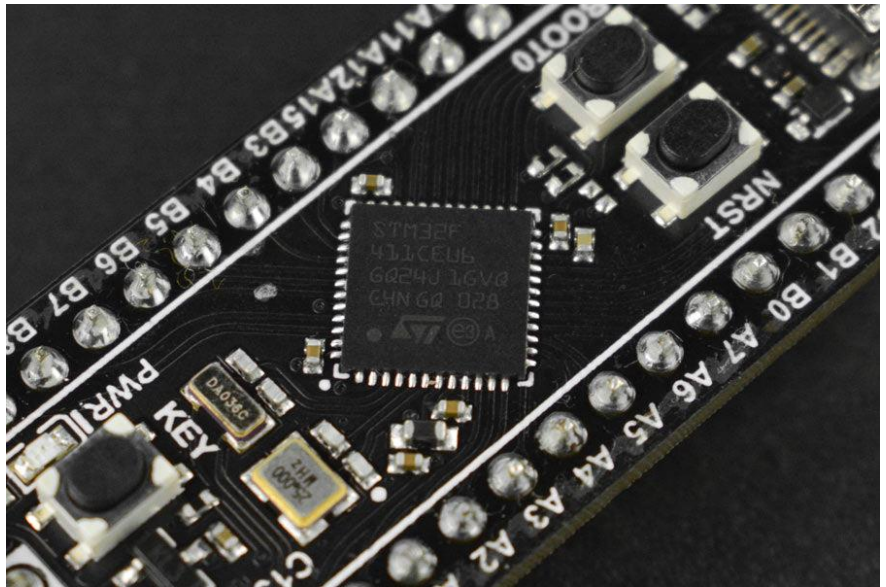
STM32F411

Microprocessor

The main focus is on processing

Usually requires external components:

- RAM
- Storage
- Peripherals



Example:

ARM Cortex-A processor

Simplified view

```
MCU:
CPU + Memory + Peripherals
        ↓
      One chip
```

```
MPU:
CPU
 ↓
External RAM
External storage
External peripherals
```

Why Use a Microcontroller?

Microcontrollers are attractive because they offer:

- Low cost
- Low power consumption
- Fast response
- Integrated peripherals
- Small size
- Deterministic behavior
- Direct hardware control

They are particularly suitable for:

- Control systems
- Sensor acquisition
- Robotics
- IoT
- Automotive systems
- Industrial systems

History of Embedded Systems

Evolution from microprocessors in the 1970s to microcontrollers, embedded OSs, ARM/SoC/Linux, IoT and connected devices, then Edge AI and autonomous systems.

A simplified evolution:

1970s : Microprocessors

↓

1980s : Microcontrollers

↓

1990s : Embedded operating systems

↓

2000s : ARM + SoC + Embedded Linux

↓

2010s : IoT + smartphones + connected devices

↓

2020s : Edge AI + robotics + autonomous systems

From Simple Controllers to Intelligent Systems

Early embedded systems often performed a simple control function.

Example:

Temperature

↓

Comparator

↓

Heater ON/OFF

Modern systems can perform:

Sensors

↓

Embedded processor

↓

Signal processing

↓

Machine learning

↓

Decision

↓

Actuators

For example, an autonomous robot can use cameras, LiDAR, IMU and other sensors, process this information, make decisions and control motors.

This evolution has increased software complexity.

That is exactly why we need good embedded software engineering.

Application Domains

Embedded systems are used in:

Automotive

- Engine control
- ABS
- Airbags
- ADAS (driver assistance systems)

Aerospace

- Flight control
- Navigation
- Communication
- Monitoring

Industry

- Robots
- Motor control
- Industrial monitoring

Application Domains : Consumer Electronics

Examples:

- Smart TVs
- Cameras
- Washing machines
- Printers
- Smartwatches
- Wireless headphones
- Routers
- Most of these devices contain multiple embedded processors.

Application Domains : Medical Systems

Examples:

- Patient monitors
- Infusion pumps
- Imaging equipment
- Wearable sensors
- Portable diagnostic devices

Important constraint

Medical systems require high:

- Reliability
- Safety
- Accuracy
- Traceability

Application Domains : IoT and Smart Systems

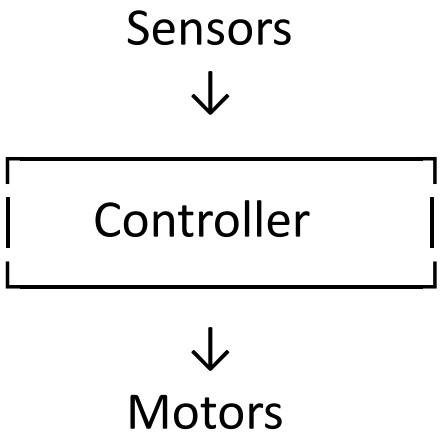
Typical architecture:

```
Sensors
   ↓
Microcontroller
   ↓
Network
   ↓
Cloud / Edge
   ↓
Application
```

Example:

```
Temperature Sensor
        ↓
ESP32
        ↓
Wi-Fi
        ↓
Server
        ↓
Web Dashboard
```

Application Domains : Robotics and Drones



A drone adds:

- IMU
- GPS
- Barometer
- Radio
- Cameras
- Flight controller

The controller cannot say:
"I will process the sensor data when I have time.“
real-time behavior.

Important

The controller must process sensor data and generate control commands within strict timing constraints.

CHARACTERISTICS OF EMBEDDED SYSTEMS

Main Characteristics

Embedded systems often have:

- A specific functionality.
- May have limited resources.
- May have real-time requirements.
- Power consumption can be very important.
- Reliability can be critical.
- Have physical size constraints and cost constraints.
- Finally, there is a strong interaction between hardware and software.

Not every embedded system has all these characteristics.

Resource Constraints

Limited Resources

Compared with PCs, many MCUs have limited:

CPU

Example: `100 MHz MCU`

RAM

Example: `128 KB RAM`

Flash

Example: `512 KB Flash`

Energy

Battery-powered systems may have strict power constraints

- An embedded **programmer** cannot simply assume that memory and CPU are unlimited.
- We have to think carefully about how much memory our program uses.
- We have to think about CPU time.
- And sometimes we even need to think about the number of bytes transmitted over a communication interface.

Why Resource Constraints Matter

Bad software can cause:

```
RAM usage too high
        ↓
Stack overflow
        ↓
```

System failure

or:

```
CPU usage too high
        ↓
Task misses deadline
        ↓
```

In a real-time system, missing a deadline can cause incorrect behavior.

Therefore embedded programming requires careful resource management.

Real-Time

- **What Is Real-Time?**
- A real-time system is a system where the **correctness of the result depends not only on the result itself, but also on when the result is produced**.

Example:

- A temperature measurement arriving 2 seconds late may be acceptable.
- A motor-control command arriving 2 seconds late may be catastrophic.

Hard vs Soft Real-Time

Hard real-time

Missing a deadline may cause system failure.

Examples:

- Airbag deployment
- Flight control
- Some industrial safety systems

Soft real-time

Missing a deadline degrades performance but may not cause complete failure.

Examples:

- Video streaming
- Multimedia
- Some monitoring systems

Deadline

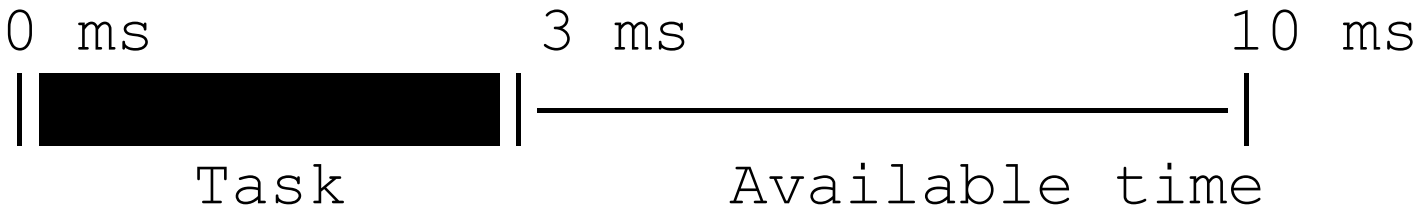
Suppose:

Task period = 10 ms

Deadline = 10 ms

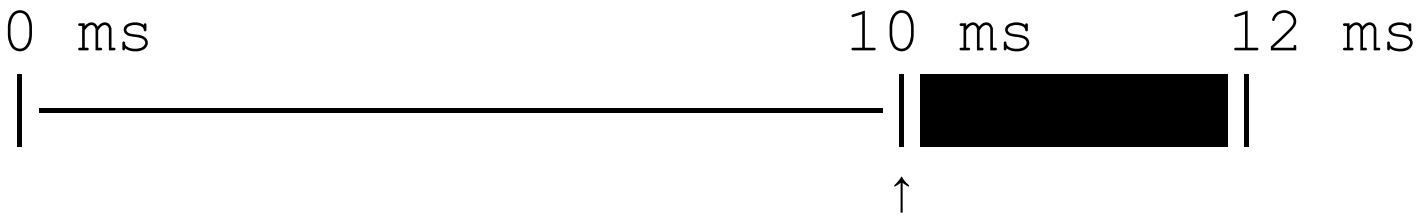
Execution time = 3 ms

Timeline:



The task finishes before its deadline.

If execution takes 12 ms:



Reliability

An embedded system may need to operate:

- For long periods
- Without human intervention
- In difficult environments

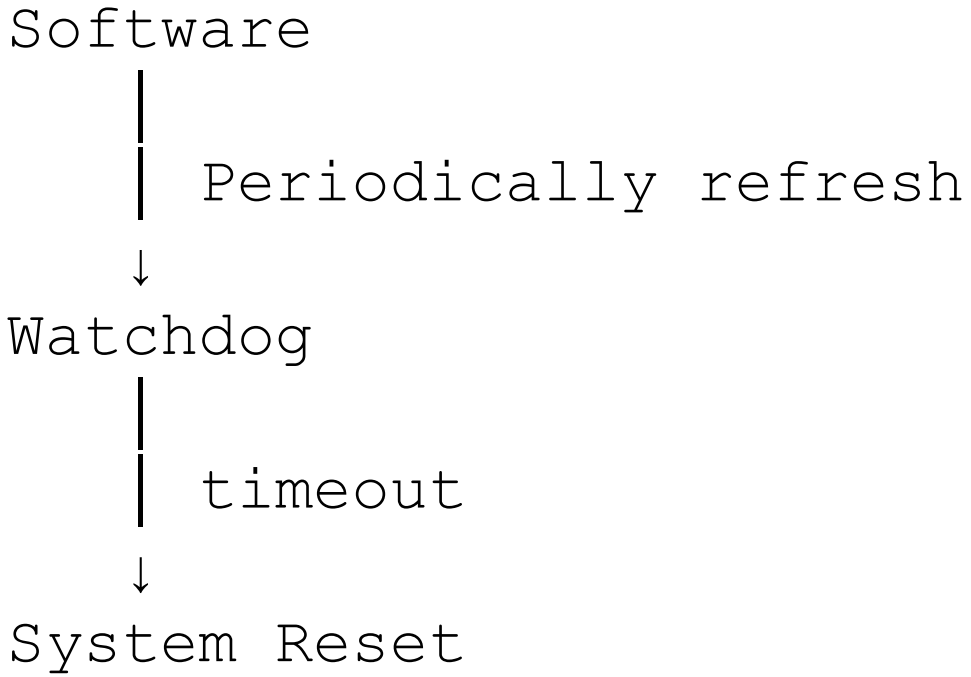
Possible techniques:

- Watchdog timer
- Error detection
- Redundancy
- Safe states
- Fault handling
- Monitoring

Watchdog Timer

A watchdog detects software failures.

Simplified:



If software stops responding, the watchdog can reset the system.

Energy Constraints

Battery-powered systems must minimize energy consumption.

Possible techniques:

- Low-power MCU modes
- Sleep modes
- Reduced clock frequency
- Peripheral shutdown
- Efficient communication
- Duty cycling

Example:

```
Active
  ↓
Measure
  ↓
Transmit
  ↓
Sleep
  ↓
Wake up
  ↓
Measure
```

Physical and Economic Constraints

An embedded product may have constraints such as:

```
Maximum PCB size
Maximum weight
Maximum manufacturing cost
Maximum power consumption
```

The best technical solution is not always the best engineering solution.
A design must balance:

Performance + Cost + Size + Power + Reliability

What Is a Sensor?

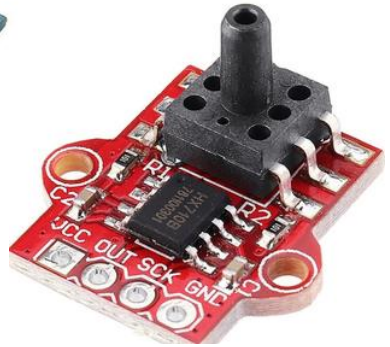
A **sensor** converts a physical quantity into a measurable signal.

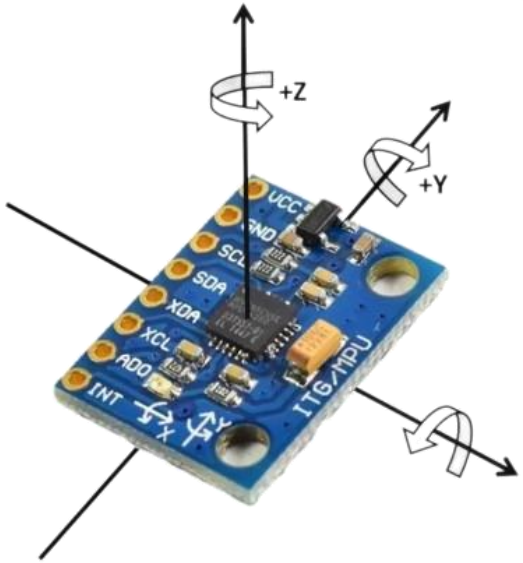
Examples:

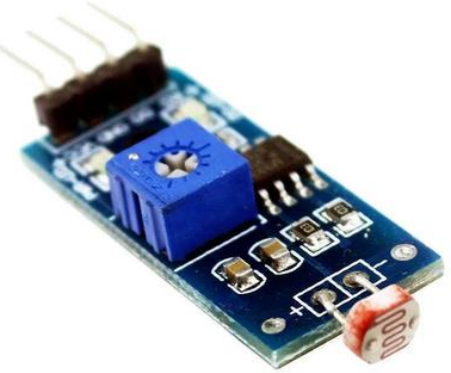
Physical quantity	**Sensor**
Temperature	Temperature sensor
Pressure	Pressure sensor
Light	Photodiode / light sensor
Acceleration	Accelerometer
Rotation	Gyroscope
Distance	Distance sensor
Humidity	Humidity sensor











Sensor Data Flow


```
Physical world
      ↓
    Sensor
      ↓
Electrical signal
      ↓
ADC / digital interface
      ↓
Microcontroller
      ↓
Software
```



The software interprets the sensor data.

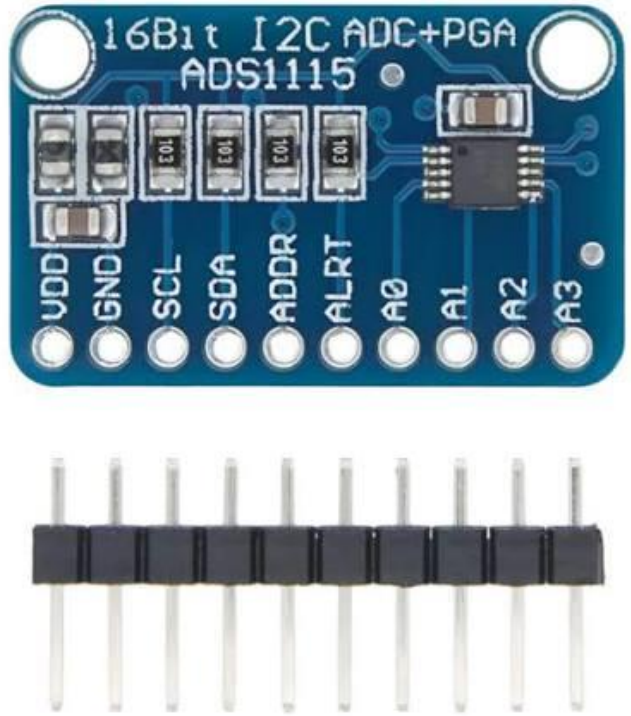
by : Dr. L.ABADA

Analog Sensor

An analog sensor may produce:

0 V ——————————— 3.3 V

The MCU uses an **ADC (Analog-to-Digital Converter)**.



Example:

Sensor

↓

Analog voltage

↓

ADC

↓

Digital value

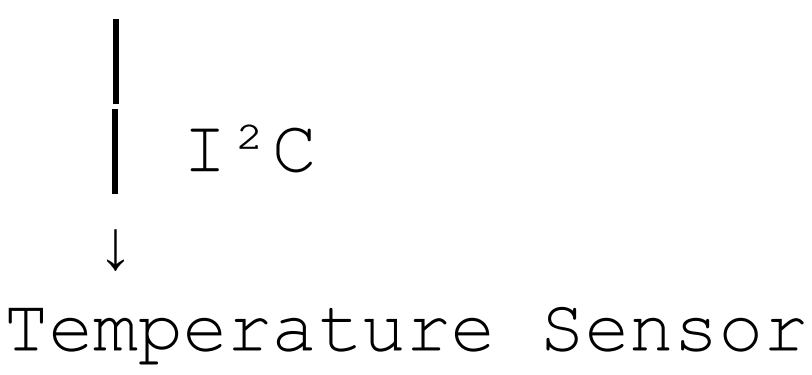
↓

Software

Digital Sensor

A digital sensor may perform the conversion internally.
Instead of giving us an analog voltage, it can communicate directly with the microcontroller.

Example:



The sensor internally performs part of the conversion.

What Is an Actuator?

An **actuator** converts a control signal into a physical action.

Examples:

- *DC motor*
- Servo motor
- Stepper motor
- Relay
- Solenoid
- Valve
- Buzzer
- LED



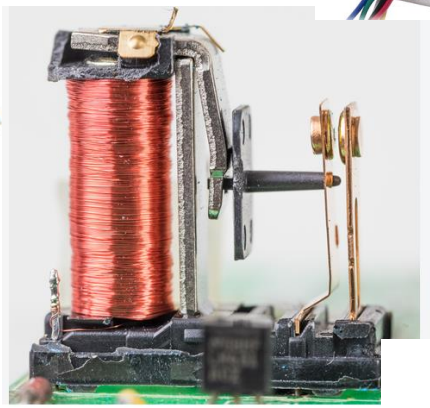














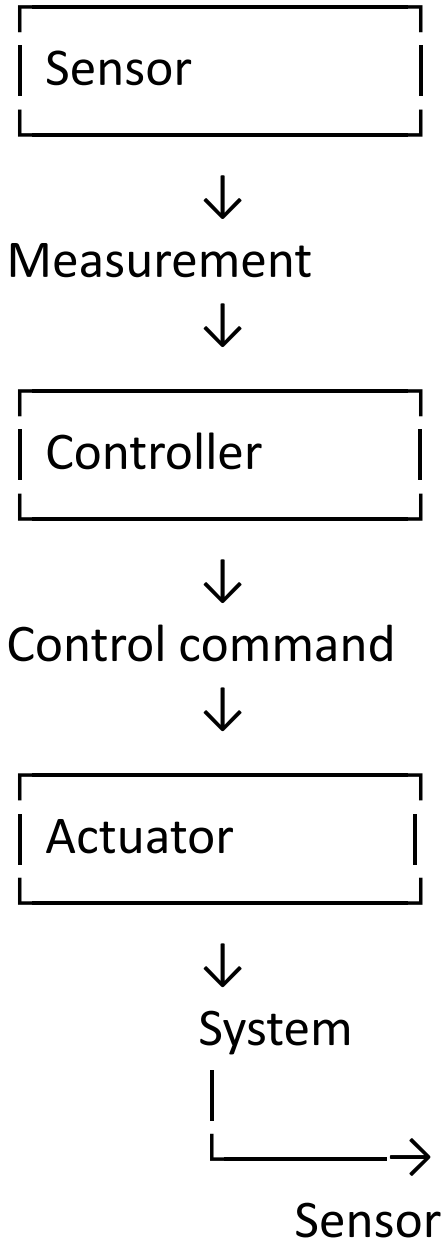






Control Loop

A basic control system:



This is a **feedback control loop**.

DATA ACQUISITION

Data acquisition means:

Collecting physical measurements and converting them into information that can be processed.

Typical chain:

Physical quantity

↓

Sensor

↓

Signal conditioning

↓

ADC / Interface

↓

MCU

↓

Software

↓

Decision / Storage / Communication

Sampling

A continuously varying signal must often be sampled.

Example:

```
Temperature:
25.1
25.2
25.3
25.2
25.4
...
```

The sampling frequency determines how often the system measures the signal.

Sampling Frequency

If a sensor is sampled every:

```
10 ms
```

then:

```
Sampling frequency = 1 / 0.010
                   = 100 Hz
```

Therefore the system obtains:

100 measurements per second.

CONTROL AND DATA TRANSFER

Embedded Data Flow

A common embedded architecture:

Sensors
↓
Data acquisition
↓
Processing
↓
Decision
↓
Control
↓
Actuators

At the same time:

Sensors → MCU → Storage
Sensors → MCU → Communication
Sensors → MCU → Display

So a modern embedded system is often doing several things at the same time: sensing, processing, controlling, communicating, and storing data.

This is one reason why embedded software can become complex.

Why Communication Interfaces?

Embedded components need to exchange data.

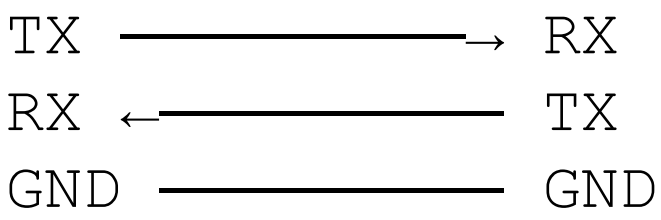
Examples:

MCU ↔ Sensor
MCU ↔ Memory
MCU ↔ GPS
MCU ↔ Radio
MCU ↔ Computer
MCU ↔ Smartphone

Different applications require different interfaces.

UART = Universal Asynchronous Receiver/Transmitter

Typical signals:



Characteristics:

- Asynchronous
- Usually point-to-point
- Simple
- Common for debugging and modules

Example:

STM32 ↔ GPS

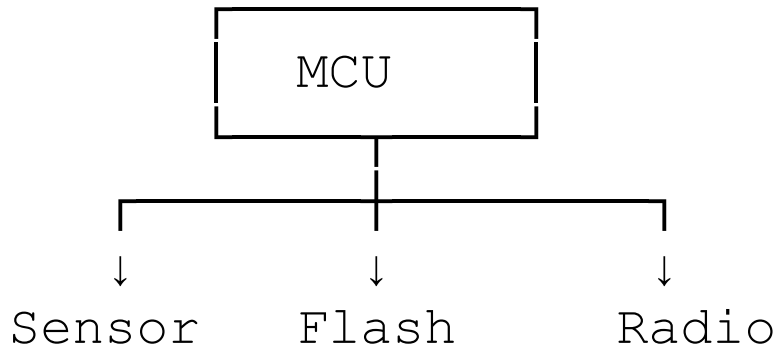
COMMUNICATION INTERFACES : SPI

SPI = Serial Peripheral Interface

Typical signals:

```
SCK
MOSI (Master Out, Slave In)
MISO (Master In, Slave Out)
CS
```

Architecture:



Advantages:

- Fast
- Simple protocol
- Full duplex

Disadvantage:

- More wires
- Usually requires a chip-select line per device

COMMUNICATION INTERFACES : I^2C

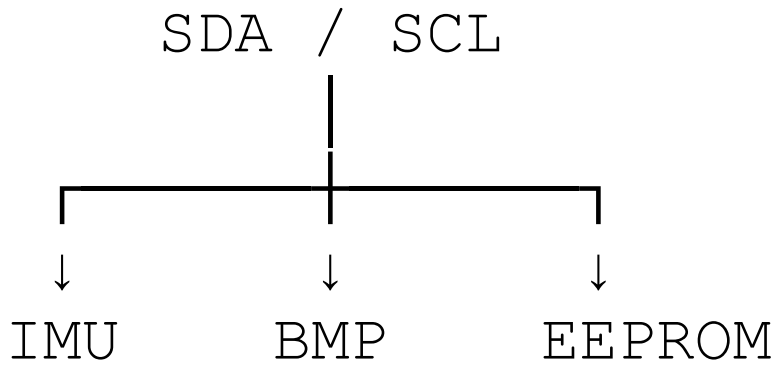
I^2C = Inter-Integrated Circuit

Uses two main wires:

SDA

SCL

Multiple devices can share the same bus.



Devices have addresses.

CAN = Controller Area Network

Widely used in:
- Automotive
- Industrial systems
- Robotics

Important characteristics:
- Multi-node bus
- Robust communication
- Message-based
- Good error handling

UART vs SPI vs I^2C

Feature	UART	SPI	I^2C
Wires	Usually 2 + GND	4+	2 + GND
Addressing	No	Chip select	Yes
Speed	Moderate	High	Moderate
Duplex	Full	Full	Half
Multiple devices	Limited	Yes	Yes
Typical use	GPS/debug	Sensors/radio	Sensors

GPS Module

A GPS receiver provides information such as:

- Latitude
- Longitude
- Altitude
- Time
- Speed
- Satellite information

Typical architecture:

```
GPS antenna
     ↓
GPS module
     ↓ UART
Microcontroller
     ↓
Data processing
     ↓
Display / control / storage
```

GPS Data Example

A GPS module may transmit textual messages such as NMEA sentences.

Example conceptually:

```
$GPRMC,...
```

The embedded software:

```
Receive
   ↓
Parse
   ↓
Validate
   ↓
Extract latitude/longitude
   ↓
Use data
```

GSM / Cellular Module

A cellular module allows an embedded system to communicate over a mobile network.

Possible applications:

- SMS
- Internet connectivity
- Remote monitoring
- Telemetry
- Tracking

Architecture:

```
Sensor
   ↓
MCU
   ↓ UART
GSM module
   ↓
Cellular network
   ↓
Server / phone
```

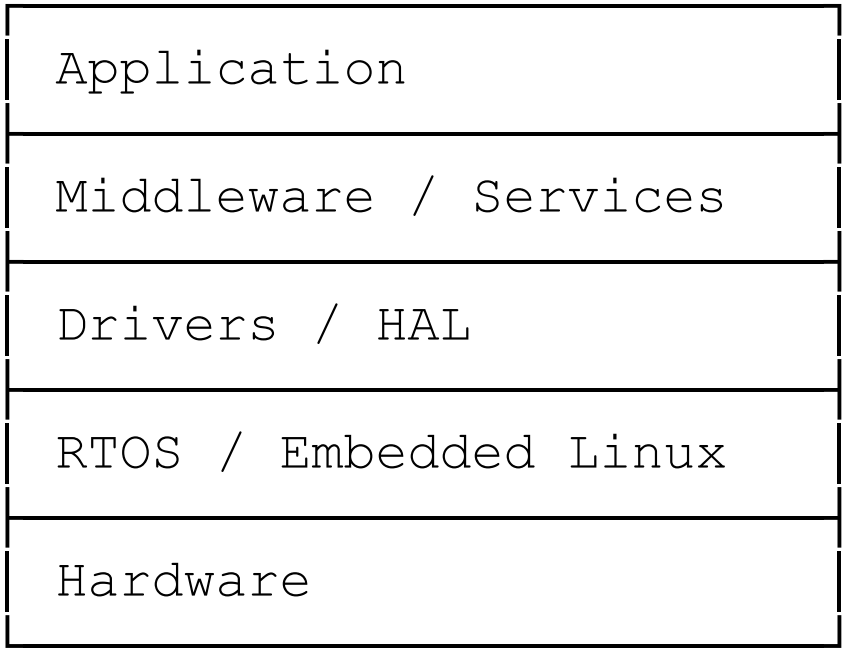
What Is Embedded Software?

Embedded software is software designed to operate a particular embedded device.

It may include:

- Startup code
- Device drivers
- Hardware abstraction
- Application logic
- Interrupt handlers
- Communication protocols
- Real-time operating system

Layers of Embedded Software



Not every embedded system contains every layer.
A small bare-metal system may be much simpler.

If I give you an STM32, a temperature sensor and a motor, what would you need to build a simple temperature-control system?